

DSA0502 – QUERY PROCESSING FOR DATA SCIENCE

CO2 - AT3 - Debugging Test

NAME: B KIRTHANAA

REG. NO - 192324244

Q1. Debug the SQLite Program

Errors:

1. college.db is not written in quotes, so Python does not treat it as a string.
2. cursor = conn.cursor is missing brackets, so the cursor is not actually created.
3. A comma is missing after id INTEGER PRIMARY KEY in the CREATE TABLE statement.
4. conn.commit() is missing, so the inserted record is not saved.
5. There is no error handling in the program.

Corrected Program:

```
import sqlite3
conn = sqlite3.connect("college.db")
cursor = conn.cursor()
cursor.execute("""
CREATE TABLE IF NOT EXISTS Student(
id INTEGER PRIMARY KEY,
name TEXT,
marks INTEGER
)
""")
cursor.execute("INSERT INTO Student VALUES (1, 'John', 90)")
conn.commit()
conn.close()
```

Explanation:

Adding quotes fixes the connect() call, adding brackets creates the cursor properly, the comma fixes the SQL syntax, and commit() saves the new record permanently.

Q2. Debug the CRUD Program

Errors:

1. The connect() and cursor() lines are joined on one line without a separator.
2. fetchone() is used inside a for loop instead of fetchall(), so it does not read all rows.
3. The UPDATE statement has no WHERE clause, so it changes every row's marks.
4. The DELETE statement has no WHERE clause, so it deletes every row.
5. conn.commit() is missing after UPDATE and DELETE.

Why every record is affected:

An UPDATE or DELETE statement without a WHERE clause applies to all rows in the table, because there is no condition to limit which rows are affected.

Corrected Program:

```
import sqlite3
conn = sqlite3.connect("college.db")
cursor = conn.cursor()
cursor.execute("SELECT * FROM Student")
for row in cursor.fetchall():
    print(row)
cursor.execute("UPDATE Student SET marks=95 WHERE id=1")
cursor.execute("DELETE FROM Student WHERE id=1")
conn.commit()
conn.close()
```

Q3. Debug the MySQL Connection**Errors:**

1. A comma is missing between user="root" and password="root".
2. There is no error handling if the connection fails.
3. The cursor and connection are never closed at the end.

Corrected Program:

```
import mysql.connector
try:
    connection = mysql.connector.connect(
        host="localhost",
        user="root",
        password="root",
        database="college"
    )
    cursor = connection.cursor()
    cursor.execute("SELECT * FROM student")
    for i in cursor:
        print(i)
except mysql.connector.Error as err:
    print("Database error:", err)
finally:
    cursor.close()
    connection.close()
```

Why the original program fails:

The missing comma between the arguments causes a `SyntaxError`, so Python cannot even run the `connect()` call.

Q4. Debug the PostgreSQL Program**Errors:**

1. The INSERT statement has three %s placeholders but only two values (1, "Ram") are given.
2. Because of this mismatch, `cursor.execute()` raises an error and the INSERT never runs.
3. There is no error handling in the program.

Corrected Program:

```
import psycopg2
conn = psycopg2.connect(dbname="college", user="postgres", password="admin")
cursor = conn.cursor()
cursor.execute(
    "INSERT INTO Student VALUES (%s, %s, %s)",
    (1, "Ram", 88)
)
conn.commit()
cursor.execute("SELECT * FROM Student")
print(cursor.fetchone())
conn.close()
```

Effect of parameter mismatch:

Every %s placeholder must have a matching value in the tuple. If the counts do not match, the database driver raises an error instead of running the query.

Q5. Multiple Errors in SQL Statements

Errors and Corrected Statements:

1. CREATE TABLE Student(id INTEGER,name TEXT marks INTEGER) - missing comma before "marks".

Corrected: CREATE TABLE Student(id INTEGER, name TEXT, marks INTEGER) [DDL]

2. INSERT Student VALUES(1,'Anu',89) - missing keyword INTO.

Corrected: INSERT INTO Student VALUES(1,'Anu',89) [DML]

3. SELECT name marks FROM Student - missing comma between column names.

Corrected: SELECT name, marks FROM Student [DQL]

4. UPDATE Student marks=90 WHERE id=1 - missing keyword SET.

Corrected: UPDATE Student SET marks=90 WHERE id=1 [DML]

5. DELETE Student WHERE id=2 - missing keyword FROM.

Corrected: DELETE FROM Student WHERE id=2 [DML]

Q6. Debug the Transaction Program

Errors:

1. The connect() and cursor() lines are joined on one line without a separator.
2. rollback() is called right after the UPDATE and INSERT, before any commit, so both changes are discarded.
3. commit() is then called, but there is nothing left to save.

Effect of rollback() and commit():

commit() permanently saves all changes made since the last commit. rollback() undoes all changes made since the last commit. Calling rollback() before commit() cancels both statements, so commit() afterward saves nothing.

Corrected Program:

```
import sqlite3
conn = sqlite3.connect("college.db")
cursor = conn.cursor()
try:
    cursor.execute("UPDATE Student SET marks=80 WHERE id=2")
    cursor.execute("INSERT INTO Student VALUES (2, 'John', 80)")
    conn.commit()
except sqlite3.Error as e:
    print("Transaction failed:", e)
    conn.rollback()
conn.close()
```

Q7. Debug the Parameterized Query

Errors:

1. id is an integer, and adding it to a string with + raises a TypeError.
2. Building the query using string concatenation is unsafe.
3. The query does not use parameter placeholders.

Why the code is unsafe:

Concatenating values directly into SQL text allows SQL injection, where a specially crafted input can change the meaning of the query. Parameterized queries keep the value as pure data.

Corrected Program:

```
id = 2
cursor.execute("SELECT * FROM Student WHERE id=?", (id,))
print(cursor.fetchall())
```

Q8. Debug the Database Fetch Program

Errors:

1. rows[20] assumes at least 21 rows exist; if not, it raises an IndexError.
2. cursor.close() is called, and then the cursor is used again for another query, which raises an error because the cursor is already closed.

Why the program fails:

It tries to use data or objects that are no longer valid — an index beyond the available rows, and a cursor that has already been closed.

Corrected Program:

```
cursor.execute("SELECT * FROM Student")
```

```

rows = cursor.fetchall()
if len(rows) > 20:
    print(rows[20])
else:
    print("Only", len(rows), "rows available.")
cursor.execute("SELECT * FROM Student")
print(cursor.fetchall())
cursor.close()

```

Q9. Integrated Debugging Problem

Errors:

1. The second INSERT reuses id=1, which already exists - raises IntegrityError.
2. UPDATE Student SET mark=95 uses the wrong column name (should be marks) - raises OperationalError.
3. SELECT ... FROM Students uses the wrong table name (should be Student) - raises OperationalError.
4. conn.commit() is missing after the CREATE/INSERT/UPDATE statements.
5. There is no try/except, so the program stops at the first error.
6. cursor.close() is never called.

Corrected Program:

```

import sqlite3
conn = sqlite3.connect("college.db")
cursor = conn.cursor()
cursor.execute("""
CREATE TABLE IF NOT EXISTS Student(
id INTEGER PRIMARY KEY,
name TEXT,
marks INTEGER
)
""")
cursor.execute("INSERT INTO Student VALUES (1, 'Asha', 90)")
cursor.execute("INSERT INTO Student VALUES (2, 'Raj', 85)")
cursor.execute("UPDATE Student SET marks=95 WHERE id=1")
cursor.execute("SELECT name, marks FROM Student")
print(cursor.fetchall())
conn.commit()
cursor.close()
conn.close()

```

Exception caused by each error:

Duplicate id -> IntegrityError | Wrong column name -> OperationalError | Wrong table name -> OperationalError | Missing commit -> silent data loss | No try/except -> program stops at first error.

Q10. Complete Database Application Debugging

Errors:

1. The program inserts into Student without making sure the table exists first.

2. UPDATE Student SET marks=95 has no WHERE clause, so it changes every row instead of just id=3.
3. id==3 uses the Python equality operator instead of the SQL operator =.
4. conn.commit() is missing after the INSERT, UPDATE, and DELETE statements.
5. There is no error handling in the program.

Corrected Program:

```
import sqlite3
conn = sqlite3.connect("college.db")
cursor = conn.cursor()
cursor.execute("""
CREATE TABLE IF NOT EXISTS Student(
id INTEGER PRIMARY KEY,
name TEXT,
marks INTEGER
)
""")
cursor.execute("INSERT INTO Student VALUES (3, 'Kiran', 90)")
conn.commit()
cursor.execute("SELECT * FROM Student WHERE id=3")
print(cursor.fetchone())
cursor.execute("UPDATE Student SET marks=95 WHERE id=3")
cursor.execute("DELETE FROM Student WHERE id=3")
conn.commit()
conn.close()
```

Predicted output:

(3, 'Kiran', 90)

After this, the UPDATE and DELETE run, so by the end of the program the Student table no longer has a row with id=3.